

#Software Engineering

- (i) Verification (during product development) Right process
- (ii) Validation (at the end) whether we develop right product

#Core activity -

- (i) Requirement analysis
- (ii) Design
- (iii) Implementation
- (iv) Testing
- (v) Maintenance

#Product

standalone system
General Product } Software Product
Customized Product }

Program = set of instructions for a specific task

Software = collection of programs + data + documents

Software Engineering = Structured approach to software development

#General Issues -

- (i) Heterogeneity
- (ii) Business & social change
- (iii) Security
- (iv) Scale

#Essential Attributes/Properties of Good Software -

- 1) Maintainability
- 2) Dependability & security
- 3) Efficiency
- 4) Acceptability

#Why Software Engineering -

- (i) Manage complexity
- (ii) Improve quality
- (iii) ~~Reusability~~ Reduce development time
- (iv) ~~Quality focus~~ Ensure maintainability
- (v) ~~System approach~~

#Core activities -

- (i) Requirement analysis
- (ii) Design
- (iii) Implementation (coding)
- (iv) Testing
- (v) Maintenance
- (vi) Project management

#Types of Prototypes

- (i) Throwaway
- (ii) Iterative

#Principles -

- (i) Modularity
- (ii) Abstraction & encapsulation
- (iii) Reusability
- (iv) Quality focus
- (v) System approach

Software Development Life Cycle (SDLC) - Goals = Efficiency & Effectiveness

Stages -

- (i) Requirement collection and analysis
- (ii) Feasibility study
- (iii) Design
 - Interface design
 - Architectural design
 - Detailed design
- (iv) Coding
- (v) Testing
- (vi) Deployment
- (vii) Maintenance

Software Testing Life Cycle (STLC) -

Steps/Level - Testing Levels -

- (i) Unit testing
- (ii) Integration testing
- (iii) System testing
- (iv) Acceptance testing

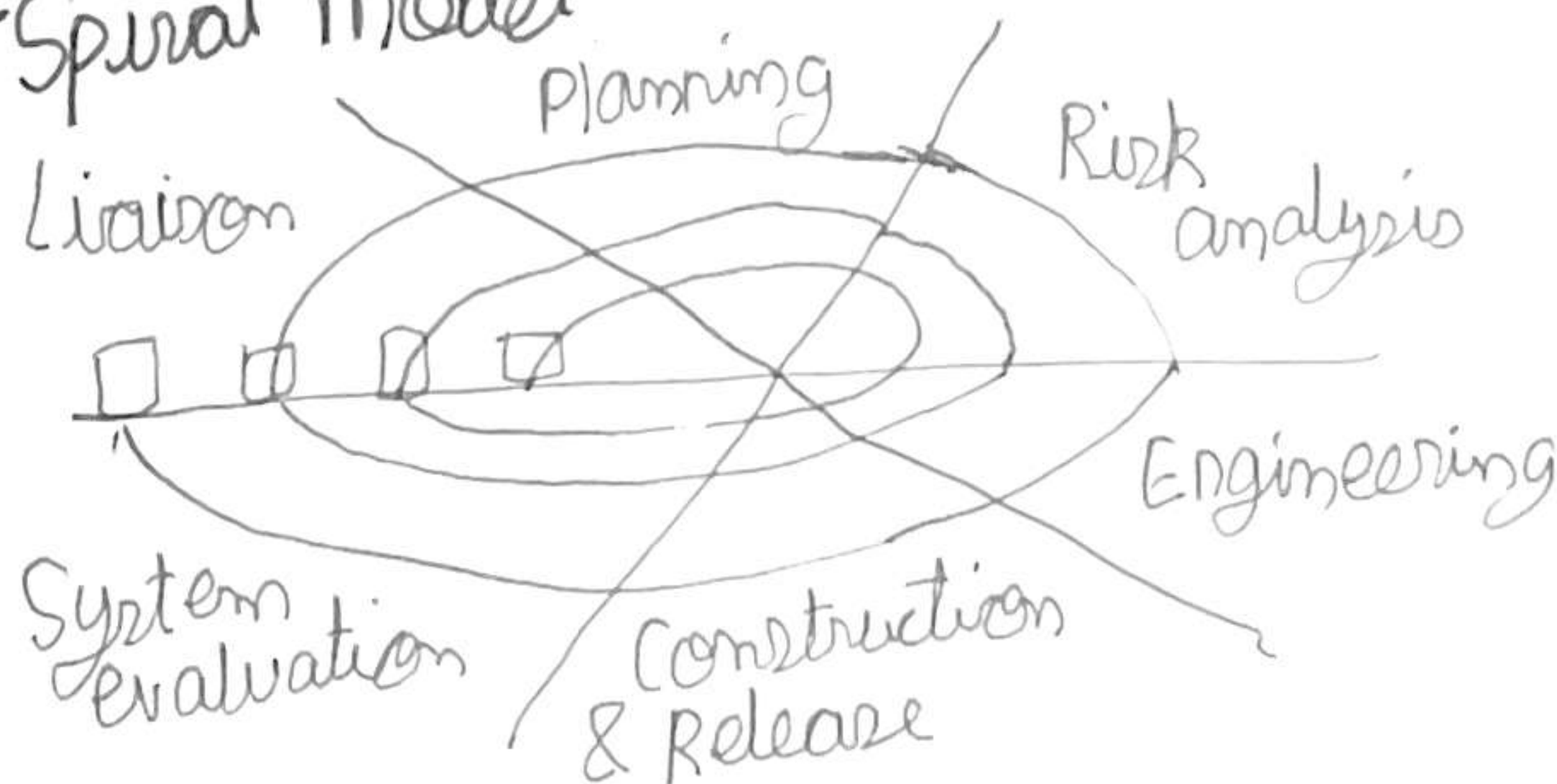
Waterfall model -

- 1) Requirement analysis
- 2) System design
- 3) Implementation
- 4) System testing
- 5) System deployment
- 6) System maintenance

Incremental model -

- Analysis → Design → Code → Test Increment 1
- Analysis → Design → Code → Test Increment 2
- Analysis → Design → Code → Test Increment 3

Spiral model -

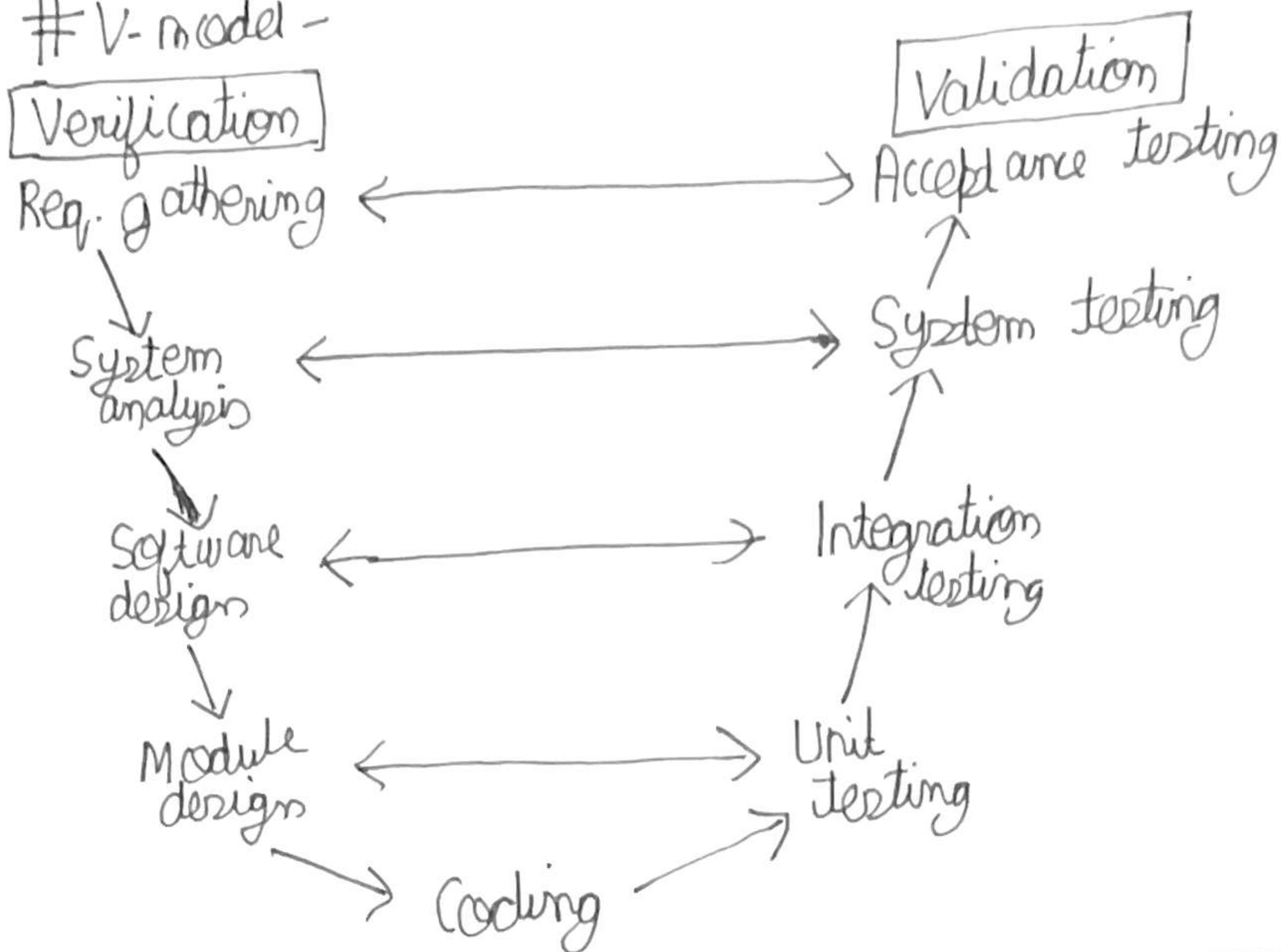


Agile Methodology

Objectives of SDLC -

- (i) Ensure that high quality systems are delivered → Built to contract (commitment)
- (ii) Provide strong management controls over the projects → Efficiency, Effectiveness, Process compliance, Cost control, Customer satisfaction
- (iii) Maximize the productivity of system staff → Built by best practices

V-model -



Top down
VS
Bottom up
(2M)

Phase 1: Requirement Collection & Analysis

Sub processes:

- Assigning Roles
- Define Project Scope
- Conduct Stakeholder interviews
- Document requirements
- Verify & validate requirements
- Prioritize requirements

Phase 2: Feasibility study

Types:

- Economic
- Legal
- Operation feasibility
- Technical
- Schedule

Phase 4: Software Development / Coding

Development

↓
Code standard

↓
Scalable code

↓
Version control

↓
Code review

Phase 6: Deployment

Activities:

- Planning software deployment
- Checking service components
- Checking target environments
- Executing deployment
- Software deployment confirmation

Phase 7: Maintenance

Types:

- Corrective software maintenance
- Preventive software maintenance
- Perfective software maintenance
- Adaptive software maintenance

#SDLC Model Comparison -

SDLC model

Advantages

Disadvantages

1) Waterfall model

- Simple & easy to understand
- Clear phases & documentation
- Suitable for small, well defined projects

- No flexibility for changes
- Testing done late
- Not suitable for complex projects

2) Incremental model

- Early delivery
- Easy to incorporate changes
- Better risk handling than waterfall model

- Requires good planning
- Overall cost may increase
- Needs well defined architecture

3) Spiral model

- Strong risk management
- Flexible to changes
- Suitable for large, complex projects

- Expensive and time consuming
- Complex to manage
- Not suitable for small projects

4) V-Model

- Early testing improves quality
- Clear verification & validation
- Works well for stable requirements

- Very rigid
- Changes are difficult
- No early prototype

#Requirement Elicitation -

- First and most crucial step of Requirement Engineering
- Systematic process of gathering information from stakeholders
- Bridges the gap between stakeholder needs and software solutions

- Why?
 - Helps ~~us~~ identify real user needs, not assumptions
 - Builds a shared understanding among developers & customers
 - Reduces project failures
 - Supports creation of SRS document
 - Saves time & cost

- Goals:

- Identify all stakeholders
- Understand business goals & workflow
- Discover functional & non functional requirements
- Reveal constraints, assumptions, and limitations
- Resolve conflicts among stakeholders
- Create a foundation for modelling analysis, and specification

DFD = Data Flow Diagram
≈ Flowchart

- Stakeholders:

- End users
 - Client / Customers
 - Business managers
 - Domain experts
 - Developers
 - Testers
 - System administrator
- } External
- } Internal

- Activities:

- Identifying stakeholders
- Understanding the business environment
- Planning elicitation techniques
- Conducting elicitation sessions
- Documenting findings
- Validating collected requirements

Requirement Elicitation Techniques -

1) Interviews -

- One on one or group
- Types:
 - Structured (pre-defined questions)
 - Unstructured (free discussions)
 - Semi structured (combination)
- Good for exploring

2) Questionnaires / Surveys -

- Used when many users are involved
- Economical and quick
- Types:
 - Open ended
 - Close ended

3) Brainstorming -

- Group creativity session
- Generates innovative ideas and solutions
- Especially useful in early project stages
- Types:
 - Passive (analyst watches)
 - Active (analyst participates)

4) Workshops / Joint Application Development (JAD) -

- Conducted with multiple stakeholders & experts
- Helps resolve conflicts
- Produces well defined requirements collaboratively

~~# Req~~

5) Document Analysis

- Examines existing:
 - Manuals
 - Forms
 - Reports
 - Previous system documentation
- Useful for understanding current problems & identifying missing functionalities

- ### # Features of a Good SRS document -
- | | |
|------------------------------|----------------------------|
| (i) Correct | (vii) Traceable |
| (ii) Complete | (viii) Ranked for Priority |
| (iii) Consistent | (ix) Design independent |
| (iv) Unambiguous | (x) Understandable |
| (v) Verifiable | |
| (vi) Modifiable / Manageable | |

Requirement Specification -

- (i) Complete
- (ii) Consistent
- (iii) Modifiable / Manageable
- (iv) Unambiguous
- (v) Verifiable
- (vi) Traceable
- (vii) Correct

Q. Requirement Analysis
VS
Requirement Specification

Requirement Validation -

- (i) Correct
- (ii) Complete
- (iii) Consistent
- (iv) Feasible
- (v) Meeting stakeholder expectations

Purpose :

- (i) Ensure agreement among all stakeholders
- (ii) Ensure that the requirement correctly describes user needs
- (iii) Detect error

Why validation is important?

- Errors found in requirements are 10-100 times cheaper to fix than errors found later
- Prevents building wrong
- To reduce time complexity
- To ensure flawless development

What is validated?

- (i) Functional requirements
- (ii) Non functional requirements
- (iii) Constraints & assumptions
- (iv) Interface requirements
- (v) Domain/business rules
- (vi) Models and diagrams (DFD, Use case, UML)
- (vii) Entire SRS document

Requirement Validation Techniques -

- (i) Informal review
- (ii) Walkthrough
- (iii) Technical review
- (iv) Formal Inspection

Output of Requirement Validation -

- (i) List of errors & inconsistencies found
- (ii) Corrected & updated SRS
- (iii) Review report
- (iv) Approval from stakeholders
- (v) Baseline SRS ready for design phase

Q. Difference between verification & validation

Verification

- 1) Are we building the products right?
- 2) Checks compliance with specification
- 3) Internal process
- 4) Ensures ~~com~~ correctness

Validation

- Are we building the right product?
- 2) Checks if it meets user needs
- 3) Involves users & stakeholders
- 4) Ensures usefulness